

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Лабораторная работа №3

по дисциплине «Проектирование защищенных интеллектуальных
информационных систем»

на тему «Анализ безопасности кода.
Часть 2. Методика оценки безопасности кода»

Выполнил студент гр. 321701

Ковальчук В. Н.
Самович В. М.

Проверил

Сальников Д. А.

Минск 2026

Цель: на основе результатов лабораторной работы №3.1 создать методику оценки безопасности кода и применить её к двум реализациям одной и той же задачи.

Задание:

- а) Создать методику оценки безопасности кода на основе результатов части 1.
- б) Сравнить по методике свою реализацию из части 1 с упрощённой.
- в) Оформить отчёт.

Постановка задачи. Готового алгоритма для сравнения в задании не было, поэтому для работы написана упрощённая реализация Lab3.2-NaiveVersion. Она решает ту же задачу, что и раздел «In-memory data» из части 1, но без мер защиты, которые были использованы там.

Описание алгоритмов

Своя реализация (часть 1). Раздел «In-memory data» приложения SecureApp: ASP.NET Core, валидация через DataAnnotations, хранилище на ConcurrentDictionary с идентификаторами через Interlocked.Increment, конфиденциальные поля шифруются AES-256-GCM с ключом из конфигурации.

Упрощённая реализация. Консольная программа Lab3.2-NaiveVersion на HttpListener с теми же операциями (создание, обновление, удаление) без авторизации. Она намеренно написана без учёта выводов части 1:

- нет валидации: любая ошибка превращается в 500 с текстом исключения;
- нет ограничения размера запроса;
- «шифрование» – XOR с ключом, записанным в коде (листинг 1);
- список activityLog, в который пишется открытый текст каждой операции и который никогда не очищается;
- обычный Dictionary и nextConfId++ без синхронизации.

```

1 string XorObfuscate(string text, bool alreadyBase64 = false)
2 {
3     var key = "my-static-secret-key-123".ToArray();
4     ...
5 }
6
7 activityLog.Add($"{DateTime.UtcNow:o} CREATE confidential: title='{title}'
   ↳ content='{content}'");
8 var id = nextConfId++;

```

Листинг 1 – Упрощённая реализация: XOR с зашитым ключом и журнал с открытым текстом

Методика оценки безопасности кода

Методика состоит из четырёх критериев. Каждый проверяется одинаково для любой реализации и даёт результат «соответствует» или «не соответствует».

а) **Невалидный ввод и размер данных.** Отправить семь запросов: неверные типы полей, сломанный JSON, пропущенное поле, пустое тело, 10 050 символов, 5 МБ, переводы строки и табуляция внутри значения. Критерий выполнен, если каждая ошибка даёт конкретный код 4xx без текста исключения, а слишком большие данные отклоняются.

б) **Параллельный доступ.** Отправить 200 одновременных запросов на создание записи (по 50 одновременно). Критерий выполнен, если получено 200 уникальных идентификаторов, 200 записей сохранено, а чтение списка после этого работает.

в) **Защита конфиденциальных данных.** По коду определить алгоритм и источник ключа. Критерий выполнен, если используется стойкий алгоритм с проверкой подлинности и случайным поппе, а ключ не хранится в исходном коде.

г) **Конфиденциальные данные в памяти.** Снять дампы после создания, обновления и удаления записи с уникальным маркером и посчитать вхождения маркера. Критерий выполнен, если после удаления в дампе нет копий, оставленных кодом приложения (структуры данных, журналы). Копии, оставленные платформой, фиксируются отдельно.

Корректность методики: проверки воспроизводимы (скрипты `compare.sh` и `dump_analysis.sh`), результат каждой проверки – число или код ответа, поэтому оценка не зависит от того, кто её проводит. Методика различает реализации: на двух реализациях разного качества она даёт разный результат (см. ниже), а критерии выведены из реальных находок части 1 (валидация, шифрование, остатки данных в дампах).

Анализ безопасности упрощённой реализации

Проверки 1 и 2 выполнены скриптом `compare.sh` (листинг 2) на свежезапущенных обеих реализациях.

```
1 == Concurrency: 200 POSTs, 50 at a time ==
2 own: unique ids 200 / 200, stored 200 / 200, GET list -> HTTP 200
3 naive: unique ids 200 / 200, stored 0 / 200, GET list -> HTTP 500
4 == Invalid input: HTTP status ==
5 case                                own    naive
6 wrong types                        400    500
7 broken JSON                        400    500
8 missing field                      400    500
9 empty body                         400    500
10 10050 chars                       400    201
11 5 MB                              400    201
12 newline, tab                      201    201
```

Листинг 2 – Невалидный ввод и параллельный доступ: своя реализация (`own`) и упрощённая (`naive`)

– **Ввод.** Упрощённая реализация на четыре разных ошибки отвечает одинаковым 500 с текстом исключения, который раскрывает внутреннее устройство сервера. Данные в 10050 символов и 5 МБ она принимает без ограничений (201). Своя реализация возвращает 400 во всех этих случаях.

– **Параллельный доступ.** После 200 одновременных записей чтение списка в упрощённой реализации падает с 500: два обычных словаря (заголовки и содержимое) рассинхронизировались. Состояние гонки недетерминировано: в другом прогоне два запроса получили один и тот же идентификатор, и одна запись была потеряна без ошибки. Своя реализация сохранила все 200 записей.

– **Шифрование.** XOR с постоянным ключом из кода обратим любым, у кого есть исходный код или исполняемый файл. Нет ни `nonce` (одинаковые данные дают одинаковый шифротекст), ни проверки подлинности.

– **Память.** Результаты дампов приведены в листинг 3.

```

1 Marker counts (UTF-8 + UTF-16 strings):
2 ./dumps/1-create.dmp: CONFIDENTIAL=13 PUBLIC=11
3 ./dumps/2-update.dmp: CONFIDENTIAL=26 PUBLIC=10
4 ./dumps/3-delete.dmp: CONFIDENTIAL=26 PUBLIC=10
5
6 activityLog lines found in 3-delete.dmp:
7
8 2026-10-08T20:12:41.3085123Z CREATE confidential: title='secret'
   ↪ content='CONFIDENTIAL-MAR
9 2026-10-08T20:12:41.7238788Z UPDATE confidential#0: title='secret'
   ↪ content='CONFIDENTIAL-M
10 2026-10-08T20:12:42.4685896Z DELETE confidential#0
11 2026-10-08T20:12:42.4685896Z DELETE confidential#0: title='secret'
   ↪ content='CONFIDENTIAL-M

```

Листинг 3 – Вхождения маркеров в дампах упрощённой реализации и строки журнала activityLog после удаления

После удаления записи число вхождений маркера в дампе не уменьшилось. Кроме буферов запроса и ответа, как и у своей реализации, в дампе остались строки activityLog с открытым текстом создания, обновления и удаления записи. Это ошибка кода приложения, и она устранима: достаточно не писать секреты в журнал.

Сравнение реализаций по методике

Критерий	Своя реализация	Упрощённая реализация
1. Невалидный ввод и размер	Соответствует: 6 из 6 ошибок дали 400	Не соответствует: 4 раза 500 с текстом исключения, большие данные приняты
2. Параллельный доступ	Соответствует: 200 из 200 записей	Не соответствует: чтение списка после гонки даёт 500
3. Защита данных	Соответствует: AES-256-GCM, попсо, ключ в конфигурации	Не соответствует: XOR, ключ в коде
4. Данные в памяти	Частично: после удаления остаются копии в буферах платформы	Не соответствует: кроме буферов остаются строки журнала с открытым текстом

Таблица 1 – Сравнение реализаций по разработанной методике

Выводы

На основе результатов части 1 разработана методика из четырёх критериев: невалидный ввод и размер данных, параллельный доступ, защита конфиденциальных данных и их остатки в памяти. По методике сравнены собственная реализация и упрощённая реализация, написанная без мер защиты.

Семантически методика показала, что безопасность кода складывается из конкретных проверяемых свойств, и каждое из них можно измерить: код ответа, число записей, алгоритм шифрования, число вхождений маркера в дампе. Своя реализация выполнила три критерия из четырёх, упрощённая – ни одного. В обеих реализациях после удаления в дампе остаются копии данных из буферов платформы; в упрощённой дополнительно остаются строки журнала, и это единственное, что устраняется кодом приложения.

Прагматически: методику можно применять к другим реализациям тех же операций, а скрипты `compare.sh` и `dump_analysis.sh` делают сравнение повторяемым.